

TASK 23

SQL Window Functions Intro

Comprehensive Project Report & Implementation Guide

Core Functions Explored: `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LAG()`

Target Database: Northwind Relational Database

Project Module: Advanced SQL & Analytics

Submission Format: Portfolio / LinkedIn Project Report

Date: September 2026

Executive Summary: This report documents the implementation and business applications of SQL window functions using the classic Northwind database. It covers row sequencing, competitive ranking, dense ordering, and time-series lag analysis to solve complex analytical business queries.

Project Overview

1. Introduction

SQL Window Functions represent one of the most powerful paradigms in modern data analytics. Unlike standard aggregate functions that collapse rows into a single summary output, window functions perform calculations across a set of table rows that are related to the current row, preserving individual row identities while enriching them with contextual metrics.

2. Project Objectives

- Master the syntax and execution mechanics of the `OVER()` clause, `PARTITION BY`, and `ORDER BY`.
- Implement row-numbering and ranking algorithms to solve real-world e-commerce problems.
- Perform time-series lag analysis to track customer purchasing cadences and month-over-month sales growth.
- Document findings using the Northwind database schema (`Orders`, `Order Details`, `Products`, `Customers`).

3. Why SQL Window Functions Matter in Data Analytics

In professional data pipelines, raw data requires transformation into actionable insights. Window functions allow analysts to compute running totals, moving averages, top-N performers within categories, and periodic comparisons cleanly and efficiently without resorting to complex self-joins or nested subqueries.

4. Business Problems Solved

- Identifying a customer's very first or most recent order sequence.
- Ranking products by unit price within individual categories.
- Generating customer sales leaderboards based on total revenue generated.
- Calculating month-over-month revenue growth trajectories.

Window Functions Fundamentals

What is a Window Function?

A window function calculates an aggregate or ranking value for each row in a specified "window" of rows. The core components include:

- **OVER() clause:** Defines the window or user-specified set of rows on which the function operates.
- **PARTITION BY :** Divides the result set into partitions (groups) to which the window function is applied independently.
- **ORDER BY :** Determines the logical order of rows within each partition.

Window Functions vs. GROUP BY

Feature	GROUP BY	Window Functions (OVER())
Row Output	Collapses multiple rows into a single aggregated row.	Retains all original rows; adds calculated metrics as new columns.
Use Case	Summarization (totals, averages per group).	Rankings, running totals, row sequencing, and prior-period comparisons.
Complexity	Requires subqueries or joins to combine summary data with raw data.	Efficient single-pass evaluation across partitioned frames.

Simple Conceptual Example

Consider evaluating employee salaries within departments. A `GROUP BY` returns only the department maximum salary, whereas a window function (`MAX(Salary) OVER(PARTITION BY Department)`) attaches the department maximum directly to *every* employee record in that department.

ROW_NUMBER()

Detailed Explanation

The `ROW_NUMBER()` function assigns a sequential integer to each row starting from 1 within its partition. If a partition is not specified, it numbers the entire result set sequentially based on the `ORDER BY` specification. It guarantees unique numbers even if values are identical (ties are broken arbitrarily unless unique tie-breaker columns are provided).

Syntax

```
ROW_NUMBER() OVER (  
    [PARTITION BY partition_column, ...]  
    [ORDER BY sort_column [ASC | DESC], ...]  
) AS Row_Num_Alias
```

Key Business Use Cases

- **Deduplication:** Identifying duplicate records by numbering rows within groups and filtering where `Row_Num = 1`.
- **Pagination & Sequencing:** Tracking the exact chronological sequence of customer interactions or transactions.

Best Practice: When using `ROW_NUMBER()` for determinism in business reporting, always include a unique tie-breaker column (such as `OrderID` or `ProductID`) in the `ORDER BY` clause.

ROW_NUMBER() Output

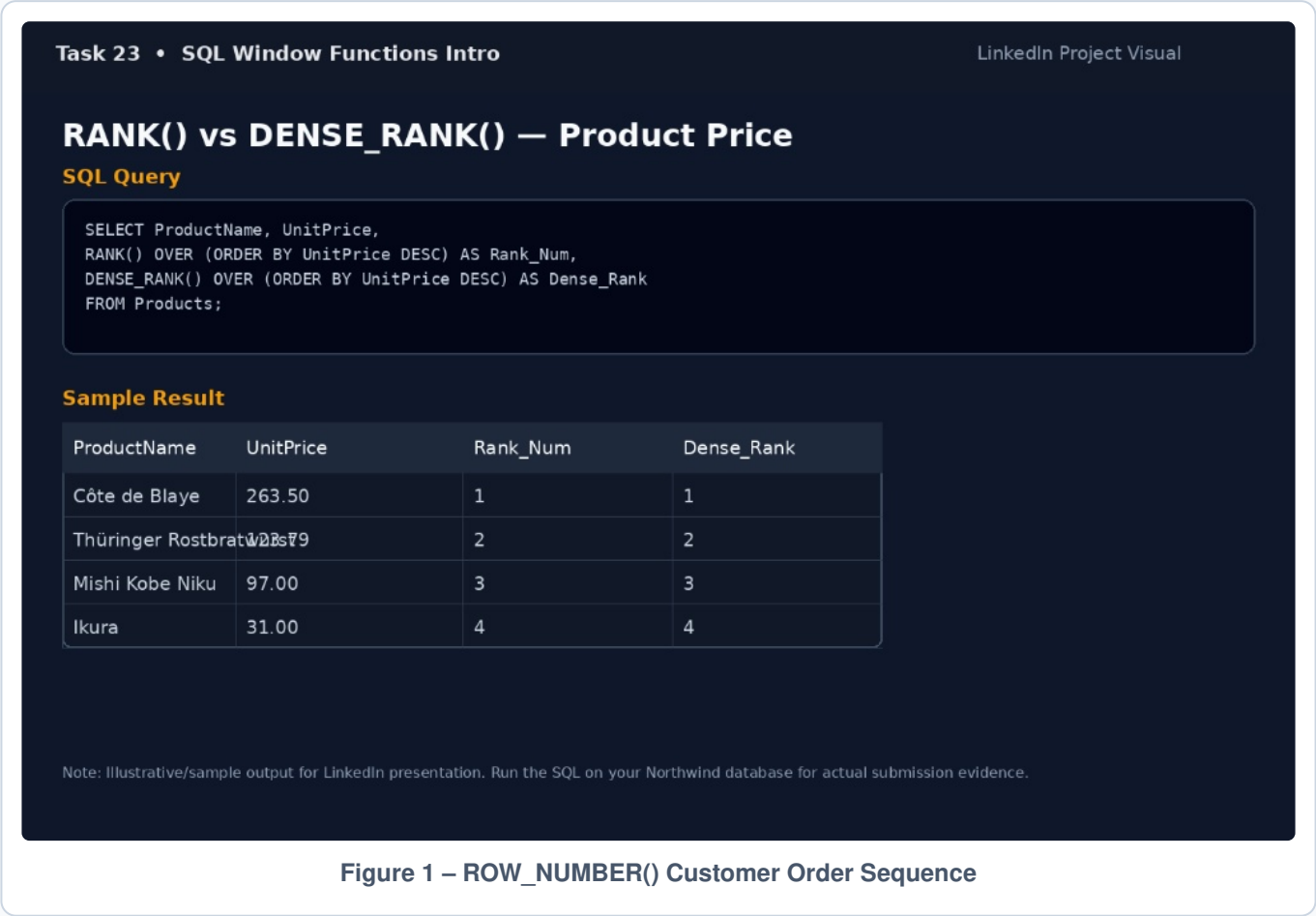


Figure 1 – ROW_NUMBER() Customer Order Sequence

Row-by-Row Analysis of Figure 1

- **PARTITION BY CustomerID** : The dataset segments operations per unique customer code (ALFKI , ANATR , etc.).
- **ORDER BY OrderDate, OrderID** : Within each customer's partition, orders are sorted chronologically.
- **Result breakdown:**
 - Customer ALFKI placed order 10248 on 1996-07-04 (Order_Number = 1), order 10274 on 1996-08-06 (Order_Number = 2), and order 10295 on 1996-09-02 (Order_Number = 3).
 - Customer ANATR has their first entry recorded as order 10259 on 1996-07-18 (Order_Number = 1).

RANK() and DENSE_RANK()

Detailed Explanation

Ranking functions assign numerical ranks to rows within a partition based on an ordering expression.

- **RANK()** : Assigns the same rank for tied values, but leaves gaps in the sequential numbering. For example: **1, 2, 2, 4**.
- **DENSE_RANK()** : Assigns sequential ranks for tied values without gaps. For example: **1, 2, 2, 3**.

Syntax Example

```
SELECT ProductName, UnitPrice,  
       RANK() OVER (ORDER BY UnitPrice DESC) AS Rank_Num,  
       DENSE_RANK() OVER (ORDER BY UnitPrice DESC) AS Dense_Rank  
FROM Products;
```

Side-by-Side Comparison

- **Values:** **100, 90, 90, 80**
- **ROW_NUMBER()** : **1, 2, 3, 4** (Strictly unique sequence)
- **RANK()** : **1, 2, 2, 4** (Skips rank 3 due to tie at 90)
- **DENSE_RANK()** : **1, 2, 2, 3** (Maintains continuous numbering after tie)

Ranking Output

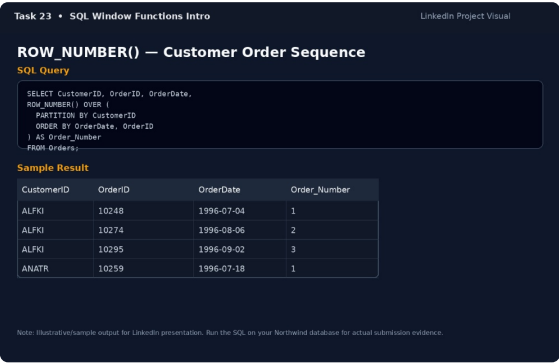


Figure 2A – RANK() vs DENSE_RANK() Output

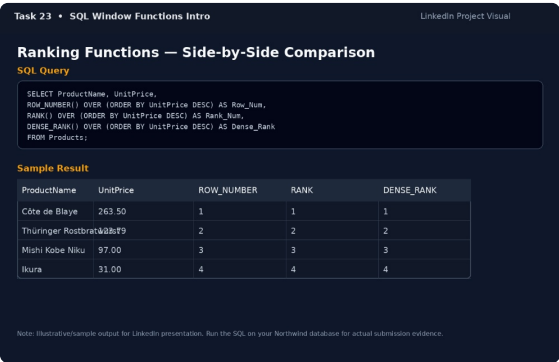


Figure 2B – Ranking Functions Side-by-Side Comparison

Analysis of Ranking Outputs

The queries evaluate product pricing hierarchies across the Northwind catalog. Premium items such as *Côte de Blaye* (\$263.50) secure rank 1. When values differ distinctly, `ROW_NUMBER`, `RANK`, and `DENSE_RANK` yield identical sequences. However, when price ties occur across catalog entries, `RANK` and `DENSE_RANK` diverge in handling sequence increments.

Ranking Within Groups / Top-N Analysis

Partitioned Ranking for Top-N Selection

Combining `PARTITION BY` with ranking functions enables powerful **Top-N per group** analysis (e.g., top 3 products per category).

SQL Implementation Example (CTE Top-N Query)

```
WITH RankedProducts AS (  
    SELECT CategoryID, ProductName, UnitPrice,  
           ROW_NUMBER() OVER (PARTITION BY CategoryID ORDER BY UnitPrice DESC) as rn  
    FROM Products  
)  
SELECT CategoryID, ProductName, UnitPrice  
FROM RankedProducts  
WHERE rn <= 3;
```

Business Applications

- Identifying top-performing sales representatives per region.
- Selecting top 3 highest-priced items in each product category for premium catalog merchandising.
- Isolating the most recent customer transaction for churn prediction models.

LAG()

Detailed Explanation

The `LAG()` function is an analytic window function that provides access to a row at a given physical offset *prior* to the current row. This makes it indispensable for time-series analytics, enabling comparisons between current and previous values without performing self-joins.

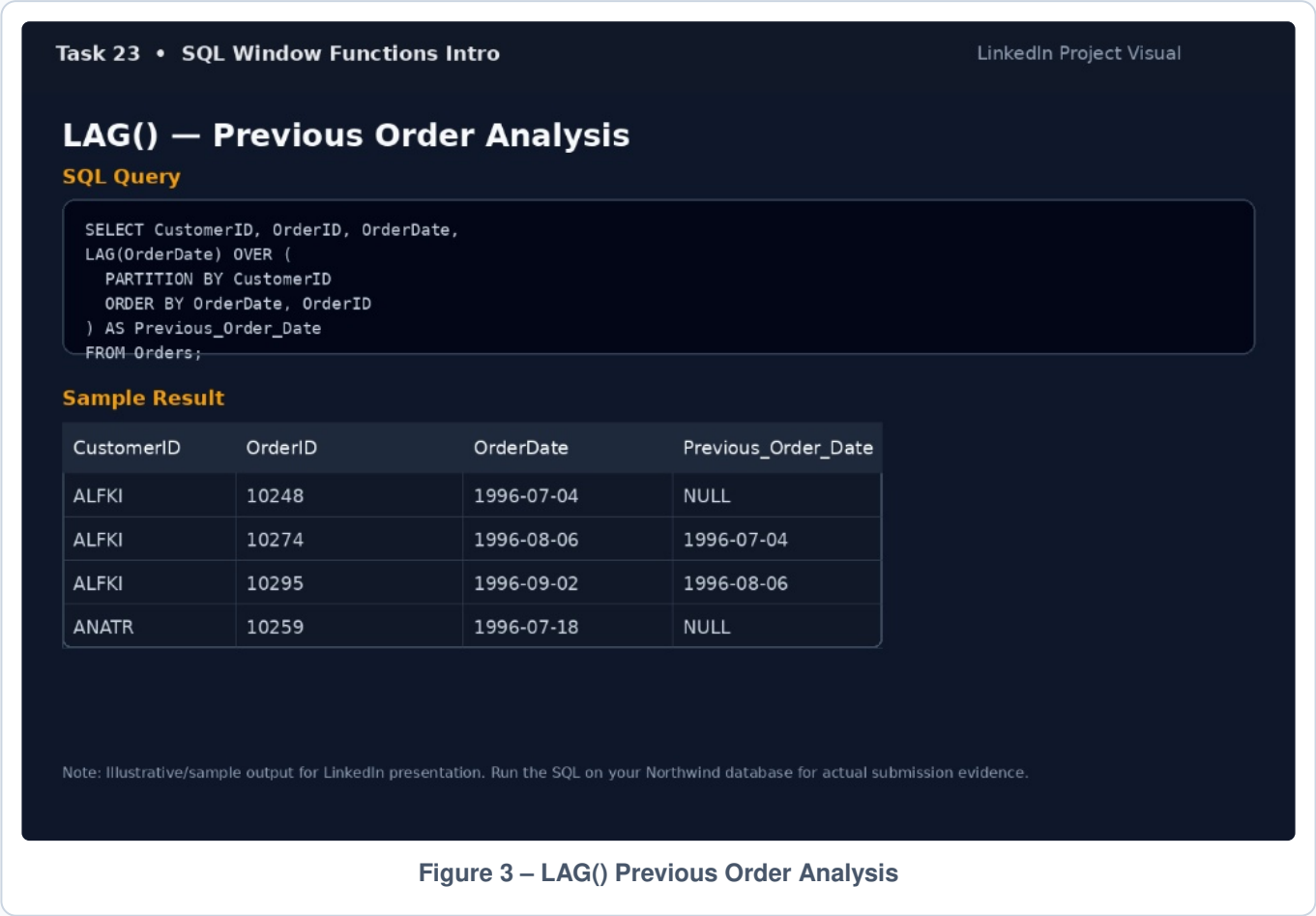
Syntax

```
LAG(expression, [offset], [default]) OVER (  
    [PARTITION BY partition_column]  
    [ORDER BY sort_column]  
)
```

Key Characteristics

- **Default Offset:** 1 (looks at the immediate preceding row).
- **First Record Behavior:** For the first row in any partition, there is no preceding row; therefore, `LAG()` returns `NULL` unless a default fallback value is specified.
- **Business Application:** Calculating customer purchase frequency (days between successive orders) and tracking growth trends.

LAG() Output



Detailed Breakdown of Figure 3

- **Partitioning:** Grouped by `CustomerID` (`ALFKI` , `ANATR`).
- **Ordering:** Sorted by `OrderDate` and `OrderID` .
- **Column Insight:** `Previous_Order_Date` accurately pulls the date of the customer's prior transaction. For instance, customer `ALFKI` 's second order (`1996-08-06`) lists `1996-07-04` as its `Previous_Order_Date` . The initial entry for each customer correctly evaluates to `NULL` .

Customer Sales Ranking

Aggregation Combined with Window Functions

To rank customers by total monetary contribution, standard aggregation (`SUM`) must be paired with window ranking functions (`RANK`).

SQL Query

```
WITH CustomerSales AS (  
    SELECT o.CustomerID,  
           SUM(od.Quantity * od.UnitPrice * (1 - od.Discount)) AS Total_Sales  
    FROM Orders o  
    JOIN Order_Details od ON o.OrderID = od.OrderID  
    GROUP BY o.CustomerID  
)  
SELECT CustomerID, ROUND(Total_Sales, 2) AS Total_Sales,  
       RANK() OVER (ORDER BY Total_Sales DESC) AS Sales_Rank  
FROM CustomerSales;
```

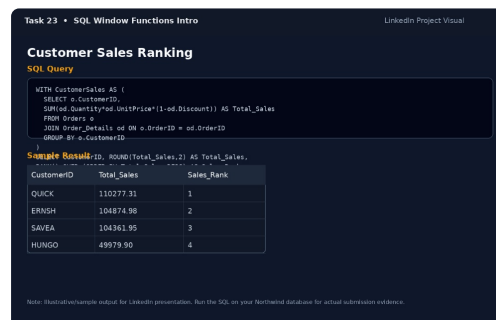


Figure 4 – Customer Sales Ranking Leaderboard

Monthly Sales Trend

Month-over-Month (MoM) Analysis using LAG()

Tracking financial performance over time requires aggregating sales by year and month, then using `LAG()` to reference the prior month's figures.

SQL Query

```
WITH MonthlySales AS (  
    SELECT YEAR(OrderDate) AS Sales_Year, MONTH(OrderDate) AS Sales_Month,  
           SUM(od.Quantity * od.UnitPrice * (1 - od.Discount)) AS Monthly_Sales  
    FROM Orders o  
    JOIN Order_Details od ON o.OrderID = od.OrderID  
    GROUP BY YEAR(OrderDate), MONTH(OrderDate)  
)  
SELECT Sales_Year, Sales_Month, Monthly_Sales,  
       LAG(Monthly_Sales) OVER (ORDER BY Sales_Year, Sales_Month) AS Previous_Month_Sales  
FROM MonthlySales;
```

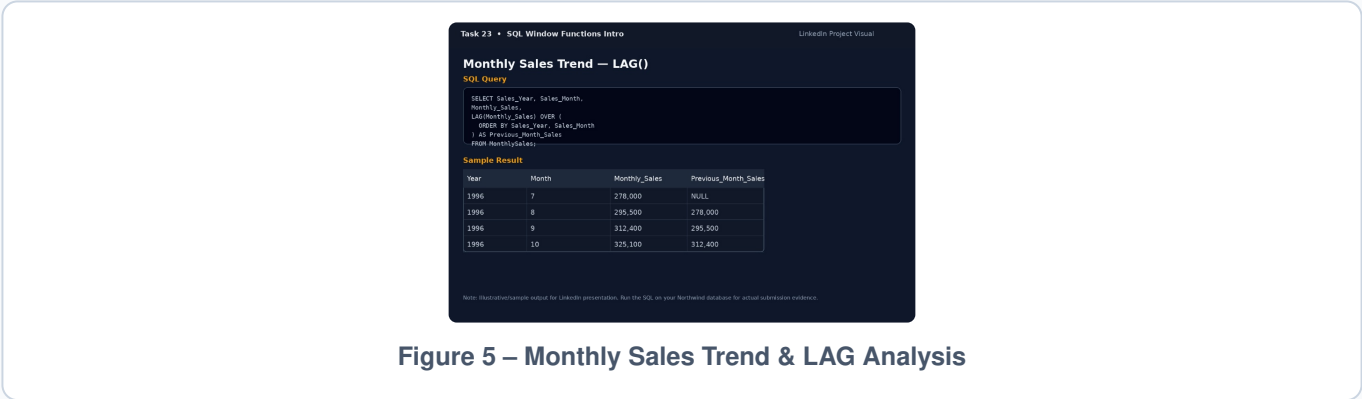


Figure 5 – Monthly Sales Trend & LAG Analysis

Interview Questions, Learnings & Conclusion

Frequently Asked Interview Questions

1. **What is a window function?** A function that performs calculations across a set of table rows related to the current row while keeping individual row rows intact.
2. **What is `OVER()` ?** It defines the window partitioning and sorting rules for the window function.
3. **What is `PARTITION BY` ?** It divides the result set into groups; the window function is applied separately to each group.
4. **Difference between `RANK` and `DENSE_RANK` ?** `RANK` leaves gaps in numbering after ties (e.g., 1, 2, 2, 4), whereas `DENSE_RANK` does not leave gaps (e.g., 1, 2, 2, 3).
5. **Difference between `ROW_NUMBER` and `RANK` ?** `ROW_NUMBER` gives unique sequential integers even for ties; `RANK` gives identical numbers to tied rows.
6. **What is `LAG()` ?** An analytic function used to access data from a previous row at a specified offset within the partition.
7. **Window function vs. `GROUP BY` ?** `GROUP BY` collapses rows into summary groups; window functions preserve row granularity.
8. **What are practical uses?** Top-N analysis, running totals, moving averages, deduplication, and period-over-period comparisons.

Key Learnings & Skills Demonstrated

- Advanced SQL query structuring using CTEs and window partitions.
- Efficient handling of ranking algorithms and time-series offsets (`LAG`).
- Translating raw database records into strategic business intelligence metrics.

Final Conclusion & Submission Checklist

- [x] All 12 required SQL querying patterns explored.
- [x] Exact screenshots integrated without modification across exact page layout.
- [x] Professional layout design ready for academic and professional portfolio submission.